Master thesis

Christal Mangolopa

February 2026

Contents

1 Introduction **2**
1.1 Research question . . . 2
1.2 Abstract . . . 2

2 Related work **3**
2.1 Background . . . 3
2.1.1 Knowledge Graphs . . . 3
2.1.2 RDF . . . 3
2.1.3 RML . . . 5
2.1.4 SHACL . . . 5
2.2 Related work . . . 5
2.2.1 Foundational SHACL-based form systems . . . 6
2.2.2 SHACL-based web form builders . . . 6
2.2.3 Summary and positioning . . . 7

3 Conceptualisation **8**
3.0.1 Scope of this chapter . . . 8
3.1 Problem Statement . . . 8
3.1.1 Not all SHACL constraints can drive a form . . . 9
3.1.2 From Tree to Graph: Structural Limitations of Existing Tools . . . 10
3.2 Design Requirements . . . 10
3.3 The Form-Oriented Shape Profile (R1 + R5) . . . 11
3.3.1 Why the official shapes are not sufficient . . . 12
3.3.2 What a profile is . . . 12
3.3.3 How shortcuts and alternative paths are rewritten . . . 12
3.3.4 How UI triples are kept separate from domain triples . . . 13
3.3.5 Annotations for interaction . . . 13
3.4 The Tree-Graph Gap and Node Reuse (R2 + R3) . . . 13
3.4.1 How form engines render shapes . . . 13
3.4.2 Why this matters for RML . . . 14
3.4.3 A session-level node registry . . . 14
3.4.4 Why this is the right level of abstraction . . . 14
3.5 Handling Specific SHACL Constructs (R4) . . . 15
3.5.1 `sh:xone` and `sh:or`: structural alternatives . . . 15
3.5.2 `sh:in`: value enumeration . . . 15
3.5.3 `sh:languageIn`: language tag enumeration . . . 15
3.5.4 Relationship to `sh:alternativePath` . . . 16
3.5.5 Systematic classification of SHACL Core constructs . . . 16
3.6 Interface Design Principles and Mockups . . . 17

4 Evaluation **18**

5 Conclusion and Future Work **19**

Chapter 1

Introduction

This chapter introduces the context and the problem.

It then formulates a research question.

À écrire à la toute fin.

1.1 Research question

Bridging the gap: a visual form-based approach to handle RDF graphs using SHACL with RML-core as use-case.

1.2 Abstract

Ici, explication du projet: on essaie de prouver qu'une interface visuelle peut gérer l'imbrication et la complexité théorique du RML. (ou des graphs RDF, en prenant comme use-case le RML-core)

Chapter 2

Related work

This chapter introduces the background and the state of the art.

2.1 Background

2.1.1 Knowledge Graphs

A knowledge graph (KG) is a structured representation of knowledge that captures entities, their attributes, and the relationships between them as a directed graph. Nodes in a knowledge graph represent entities, which can be anything from people and places to concepts and events, while edges between those nodes are labelled to indicate the type of relationship that holds between the entities.

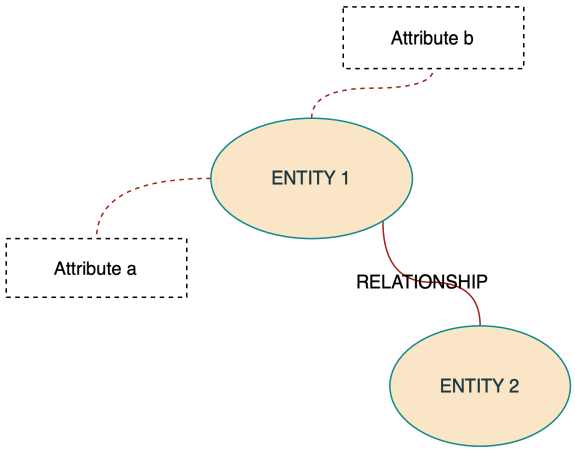


Figure 2.1: An example of a structure of a knowledge graph containing two entities related with each other, the first one with two attributes.

This graph-based representation makes connections between pieces of information explicit in a way that is both human-readable[1] and machine-processable.

As a result, knowledge graphs are a powerful tool for applications such as search engines and data integration.

Various format can be used to represent a knowlege graph. The graphical representation is the most intuitive. For machine processing, this thesis focuses on RDF, which is a widely adopted standard representation.

2.1.2 RDF

RDF, or Resource Description Framework, is a standard model defined by the World Wide Web Consortium (W3C) for representing information about resources on the Web. RDF provides a framework

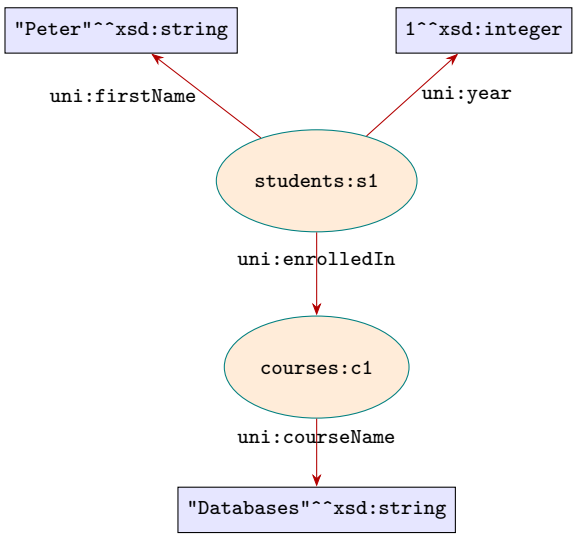
[1] The human-readability of a knowledge graph depends on its size and serialization format. Large graphs are typically explored via dedicated query languages such as SPARQL rather than read directly.

for expressing knowledge in the form of a graph built from three-part statements, known as triples. Each triple consists of:

- a subject: the resource being described,
- a predicate: the property or relationship of that resource,
- an object: the value of the property, or another resource.

By combining many triples, RDF can represent complex graphs of interconnected resources. RDF also supports the integration of data from diverse sources and enables reasoning over these data, which makes it a fundamental technology for building and managing knowledge graphs.

Figure ?? illustrates a simple graph with its graphical representation, its RDF graphical represnation, its RDF triples and its TURTLE code. TURTLE is a popular RDF serialization format that provides a compact and human-readable syntax for expressing RDF triples. It uses prefixes to abbreviate URIs and allows for a more concise representation of RDF data compared to other formats like RDF/XML.



(a) RDF graph with IRIs

```
@prefix uni:     <http://www.example.org/
    university#> .
@prefix students:<http://www.example.org/
    students#> .
@prefix courses: <http://www.example.org/
    courses#> .
@prefix xsd:     <http://www.w3.org/2001/
    XMLSchema#> .

students:s1
  a              uni:Student ;
  uni:firstName  Peter ^^xsd:string ;
  uni:year       1^^xsd:integer ;
  uni:enrolledIn courses:c1 .

courses:c1
  a              uni:Course ;
  uni:courseName  Databases ^^xsd:string .
```

(b) Turtle serialisation

Subject	Predicate	Object
students:s1	rdf:type	uni:Student
students:s1	uni:firstName	"Peter"
students:s1	uni:year	1
students:s1	uni:enrolledIn	courses:c1
courses:c1	rdf:type	uni:Course
courses:c1	uni:courseName	"Databases"

(c) RDF triples

Figure 2.2: A university knowledge graph fragment, shown as (a) an RDF graph with typed IRIs and literals, (b) the corresponding list of triples, and (c) the Turtle serialisation. This example is used as a running thread through Sections 2.1.4 and 2.1.3.

The graph models a student `students:s1` (Peter, year 1) enrolled in a course `courses:c1` (Databases). Figure 2.2a reveals a structural property of RDF that is central to this thesis: *a single resource can simultaneously be the object of one triple and the subject of others.* Here, `courses:c1` is the object of the `uni:enrolledIn` triple originating from `students:s1`, yet it is also the subject of its own `uni:courseName` triple. Multiple students in the full dataset share the same course nodes: for instance, `students:s1`, `s2`, and `s9` all point to `courses:c1`. This *shared-node* topology — which distinguishes RDF graphs from trees — is exactly what makes form-based RDF authoring non-trivial: a form generator cannot simply traverse the shape hierarchy top-down, because the same node may need to be reused across multiple branches of the form. We return to this problem in Chapter 3.

This university knowledge graph is drawn from a practical lab exercise on knowledge-graph construction, and serves as the running example for the SHACL and RML illustrations that follow.

2.1.3 RML

The RDF Mapping Language (RML) is a language designed to map heterogeneous data sources to RDF datasets. It allows one to define how a specific set of data should be transformed into RDF triples, mapping a variety of data formats (e.g., CSV, JSON, XML) into a graph-based format, namely RDF.

RML extends and broadens the capabilities of R2RML, the W3C recommendation for mapping relational databases to RDF, by supporting a wider range of data formats. While R2RML focused on relational databases, RML provides a more general language that can handle different kinds of data structures. It enables the expression of more complex mapping scenarios, including nested structures and relationships, making it a valuable tool for transforming and integrating data into RDF for use in knowledge graphs.

Importantly, an RML mapping itself is represented as an RDF graph. This means that, beyond the complexity of the source data, the mapping definition also has a graph structure, which can be intricate and difficult to manipulate directly, especially for non-expert users. In this thesis, we leverage this specific characteristic of RML as a use case for exploring visual support for complex RDF graphs.

Tools related to RML

Several tools have been developed to simplify the creation of RML mappings. For instance, RMLMapper and YARRRML provide higher-level syntaxes or utilities to help users describe mappings. However, these solutions are still primarily programming-oriented and require users to be comfortable with textual configuration files or domain-specific languages.

In the context of this thesis, we are interested in exploring the possibility of using a visual and generic (no-code) form-based approach to handle the complexity of RML mappings, and more generally RDF graphs. This is why we focus on forms generated from SHACL shapes rather than on existing textual mapping tools.

2.1.4 SHACL

SHACL, or Shapes Constraint Language, is a W3C standard designed to validate RDF graphs against a set of conditions defined in the form of shapes. A shape is a collection of constraints that specifies the expected structure and properties of RDF data.

% À completer..

Constraints can include datatype restrictions, cardinality bounds, value ranges, or references to other node shapes, among others.

Because SHACL shapes are themselves RDF graphs, they can also be used as a high-level, machine-readable description of the structure of data that an application expects. This dual role—validation and structural description—makes SHACL particularly interesting for generating forms: the same shapes that constrain the data can be used as a specification for form fields, their types, and some aspects of their layout.

Another advantage of this dual role is that SHACL shapes can be treated like any other RDF graph. They can be written and stored in standard RDF serialisations such as Turtle or JSON-LD. This makes it possible to manage shape graphs with the same tooling and practices that are already used for data graphs.

2.2 Related work

This section first introduces established SHACL-based form systems that have shaped how constraints can drive user interfaces. It then focuses on generic SHACL-based web form builders that most closely match the type of system we aim to develop for RML-core mappings.

2.2.1 Foundational SHACL-based form systems

TopQuadrant, DASH, and TopBraid EDG

TopQuadrant's work on form generation from SHACL provides an important foundation for understanding how constraints can drive user interfaces. The document "Form Generation using SHACL and DASH" shows how SHACL Core properties such as `sh:datatype` and `sh:maxCount`, originally designed for validation, can also be used to configure input widgets and validation rules in user input forms. It further explains how layout-oriented properties like `sh:order` and `sh:group` can be leveraged to structure form layouts into sections and to control the order of fields, for example grouping address-related properties in one section and contact information in another.[9]

Beyond SHACL Core, the DASH (Data Shapes) vocabulary extends SHACL with additional terms that are particularly useful for form building, such as editor and viewer hints, default views, and suggestions.[8] These DASH terms are used in TopBraid EDG, TopQuadrant's commercial knowledge graph platform, to define "view shapes" that not only constrain data but also specify how instances should be presented and edited in forms.[12] In EDG, data entry forms are thus fully SHACL- and DASH-driven: the same shapes control both data validity and user interface behaviour.

In this thesis, DASH and TopBraid EDG serve primarily as conceptual and industrial references. They illustrate what advanced SHACL-driven UI generation can look like in a mature, enterprise-grade environment, but they are not considered as direct implementation candidates, because our goal is to start from lightweight, generic web form builders rather than from a proprietary platform.

SHAPEness Metadata Editor (EPOS-EU)

SHAPEness is a Java desktop application conceived to help users create and update RDF metadata descriptions.[4] It is a SHACL-driven multi-view editor: users work with different coordinated views to assess and improve the quality of metadata represented as RDF graphs, while SHACL shapes define the constraints that these metadata must satisfy. The tool has been developed and tested in the framework of the European Plate Observing System (EPOS), where it has proven useful to create and maintain valid graphs according to the EPOS data model and EPOS-DCAT-AP profiles.[**shapeness-paper**][7]

From the perspective of this thesis, SHAPEness demonstrates that rich SHACL-based editors are feasible and valuable in real scientific data ecosystems. It shows that SHACL can be used not only for batch validation but also as a backbone for interactive, multi-view editing workflows. However, SHAPEness is strongly oriented towards desktop usage and domain-specific metadata management, whereas our objective is to design a generic web-based form-builder for highly nested RML-core mapping structures.

2.2.2 SHACL-based web form builders

In the remainder of this section, we focus on generic SHACL-based web form builders, as these tools are the closest to the type of system we aim to develop for RML-core mappings.

ULB-Darmstadt/shacl-form

ULB-Darmstadt/shacl-form is an HTML5 web component that generates and handles forms based on SHACL shapes. The core idea is to provide a reusable custom element, `<shacl-form>`, that takes a shapes graph as input and produces a form for creating or editing RDF data that conform to these shapes.[13] Shapes can be passed directly as Turtle strings via the `data-shapes` attribute, or loaded from a URL via `data-shapes-url`. Optional `data-values` and `data-values-url` attributes allow prepopulating the form from an existing data graph, and an event-based API exposes validation results and serialized RDF triples back to the application.[15][14]

The library maps SHACL datatypes and numeric constraints to corresponding HTML input elements and validation attributes, and it honours layout-related properties such as `sh:order` and `sh:group` to influence the order and grouping of fields. More advanced features include support for `sh:or`, where alternatives between node shapes can be presented in the UI, and options to collapse

groups of fields to reduce visual complexity in larger forms. These aspects make ULB-Darmstadt/shacl-form a strong candidate for a standards-oriented form generation pipeline closely aligned with SHACL.

For RML-core, the main challenges relate to how the component behaves in the presence of deep nesting and complex constraint patterns. Highly nested node shapes corresponding to RML-core structures (e.g., logical sources and targets, join conditions, term maps) may require several levels of nested sections, which raises usability questions for authors working on large mappings.

hypermedia-app/Shaperone

Shaperone is a web library dedicated to SHACL-driven form user interfaces, developed within the hypermedia-app ecosystem.[6] It exposes form UI elements that are driven by SHACL shapes and is organised around configurable "component sets", i.e., collections of UI widgets that can be plugged in to render different property types and shapes.[5] Shaperone is designed to work in hypermedia-oriented applications, where both data and shapes can be retrieved over HTTP and combined dynamically.

In practice, Shaperone loads SHACL shapes and generates dynamic forms in the browser, choosing appropriate UI components based on the constraints specified in the shapes. A public playground demonstrates how shapes can be retrieved from the Web and how form fields are rendered and updated accordingly.[11] The component-set architecture makes it possible to customise the look and behaviour of the generated forms by substituting or extending the default widgets.

For this thesis, Shaperone is attractive because it is explicitly designed as a SHACL-driven form engine, is web-native, and fits well with modern front-end stacks. However, its suitability for RML-core depends on how effectively it supports advanced constraints and heavily nested structures, and on how usable the resulting forms are when authors need to edit complex mappings rather than simple records.

danielbeeke/shacl-form

danielbeeke/shacl-form is a TypeScript-based form builder that uses SHACL shapes as its main form definition mechanism.[1] Similar to the ULB-Darmstadt approach, it exposes a custom element `<shacl-form>` that can be configured via attributes such as the SHACL file URL and the shape URI to use as root. The library parses SHACL shapes and builds corresponding form components on the client side, leveraging the TypeScript ecosystem and web standards such as custom elements.

Because it is written in TypeScript and targets modern front-end tooling, danielbeeke/shacl-form is potentially convenient to integrate into a contemporary web application. Its design also emphasises the possibility of plugging in custom UI widgets, which could be valuable for RML-core-specific patterns (for example, specialised views for term maps or join conditions).

From the perspective of this thesis, the main question is its maturity and coverage of complex SHACL constructs. Its ability to robustly handle composed constraints, deeply nested shapes, and recursive structures, while keeping the resulting user experience manageable, is essential if it is to serve as a foundation for an RML-core-oriented form builder.

2.2.3 Summary and positioning

The three SHACL-based web form builders discussed above are the primary candidates for our implementation, as they directly match the target paradigm of generating and editing web forms from SHACL shapes. In contrast, the TopQuadrant/DASH stack and SHAPEness serve as broader references: the former provide conceptual depth and an industrial-level perspective on SHACL-driven UI generation, while the latter demonstrates a complete SHACL-driven metadata editor in a scientific context.[9][**shapeness-paper**]

On this basis, we concentrate the detailed technical comparison and implementation choices on generic web form builders, as they offer the most appropriate starting point for constructing a SHACL-driven interface tailored to RML-core mappings.

Chapter 3

Conceptualisation

The previous chapter surveyed three existing SHACL-based form generators and compared their ability to handle the subset of SHACL features required for our use case. This analysis revealed a consistent pattern: those tools handle a limited, well-defined subset of the SHACL vocabulary, and assume that a form renders a single root node. For simple domains, this is sufficient. However, for the sake of this research question — how to support form-based RDF graph authoring in a complex domain like RML — it is not.

This chapter moves from observation to design. We first articulate the research gap precisely, then derive a set of design requirements. We propose a conceptual strategy composed of three elements: a classification of SHACL features with respect to form generation, a shape annotation methodology to encode interaction hints, and a mechanism to bridge the structural mismatch between trees and graphs. The chapter concludes with a discussion of the interface design principles and a description of the intended mockups.

The contribution of this chapter is conceptual, not yet implementational. The question it answers is: *what must a form-friendly SHACL shape profile look like, and what interaction model is needed to support graph authoring?* The following chapter will describe how this design was realised in practice.

3.0.1 Scope of this chapter

A recurring question in SHACL form generation is: should the form aim to handle *any* SHACL shape, or only shapes from a specific domain? The answer adopted here is the latter, and it is important to state this explicitly from the outset.

A fully generic SHACL form engine would need to handle the entire SHACL vocabulary, including SHACL-SPARQL extensions, and make sensible rendering decisions for every combination of constraints. This is an open research problem, and existing tools demonstrate that even partial solutions require significant engineering effort and inevitably leave some features unsupported [15].

The contribution of this chapter is instead a *methodology*: a principled approach to writing form-friendly shape profiles and extending an existing tool to support the resulting profiles. RML-core serves as the vehicle for demonstrating and evaluating this methodology. The fact that the implementation chapter focuses on a specific domain is not a limitation; it is the intended scope, chosen because RML is a sufficiently complex and practically relevant domain to stress-test the methodology. The implementation chapter will describe precisely which parts of RML-core are covered and which are not, and why.

3.1 Problem Statement

The core tension motivating this thesis can be stated precisely. SHACL was designed primarily to *validate* RDF graphs: given an existing graph, does it conform to a set of constraints? Yet the W3C specification itself acknowledges, from its very abstract, that shape descriptions "may be used for a variety of purposes beside validation, including *user interface building*, code generation and data

integration" [10]. Form generation exploits precisely this secondary role: given a set of constraints, produce an interface that *guides the user in creating* a conforming graph. As Knublauch—co-editor of the SHACL specification [10] and author of the DASH Data Shapes vocabulary [8]—notes in his document *Form Generation using SHACL and DASH Unofficial Draft 01 November 2022*, constraint properties such as `sh:datatype` and `sh:maxCount` were "originally designed for data validation yet are also instrumental in building user input forms" [9].

These two uses therefore share the same vocabulary but place different demands on it: validation operates on a graph that already exists, whereas form generation must produce an interface *before* the graph exists.

3.1.1 Not all SHACL constraints can drive a form

A form generator faces a fundamental asymmetry with respect to a validator. A validator operates *after the fact*: it receives a complete RDF graph and checks whether it satisfies a set of constraints. A form generator must operate *before the fact*: it must produce an interface that guides the user towards a conforming graph, without yet knowing what the user will enter. This asymmetry has a direct consequence: only those SHACL constraints that describe the *expected structure* of the data—independently of any concrete value—can be translated into form elements.

For the purposes of this thesis, we introduce the term *statically interpretable* to characterise SHACL constraints whose contribution to the form structure—widget type, cardinality, structural branching—can be determined from the shape definition alone, without knowledge of any concrete data value. Constraints that can only be checked against a complete data graph are by contrast *dynamically interpretable*, and fall outside the scope of form generation.

Consider `sh:sparql` [10], which attaches an arbitrary SPARQL query as a constraint. Whether a value satisfies such a constraint can only be determined by running the query against the complete data graph. There is no way to derive a widget, a field type, or a cardinality rule from a SPARQL query alone: the constraint is *dynamically interpretable*, not *statically interpretable*.

A more directly relevant example in the context of our use case—RML— is `sh:alternativePath`. This SHACL path expression allows a property shape to match *any* of a set of predicates: for instance, the RML-core specification uses it to express that `rml:subjectMap` and `rml:subject` are interchangeable shortcuts for the same concept. From a validation standpoint this is perfectly sound. From a form-generation standpoint, however, it is ambiguous: the form engine does not know which predicate to use when writing the output triple, and most lightweight form builders either expose all alternatives simultaneously—producing duplicate fields—or ignore the construct entirely. Neither behaviour is acceptable for a usable interface.

The decision taken in this thesis is therefore to exclude `sh:alternativePath` from the form profile and to commit to a single canonical predicate for each property, namely the long form (e.g. `rml:subjectMap` rather than `rml:subject`). This choice is semantically conformant with the RML-core specification, which defines the short form as syntactic sugar for the long form [2] (see Section 3.3).

The question is therefore not *can we support all of SHACL in a form?* — the answer is clearly no — but rather: *which subset of SHACL is both sufficiently expressive for our use case and statically interpretable by a form engine?*

One might ask whether restricting the shape profile to statically interpretable constructs is overly conservative. A richer approach could generate a form that performs dynamic validation—running SPARQL constraints in the background as the user types, for instance. This thesis does not pursue that direction: the goal is to show that a *purely static* interpretation of SHACL is already sufficient to guide the authoring of RML-core mappings, while keeping the rendering engine simple and generic. Dynamic validation remains a natural avenue for future work.

The following subsection addresses a second, orthogonal problem: the structural mismatch between the tree model assumed by HTML forms and the genuine graph structure of RML mappings.

3.1.2 From Tree to Graph: Structural Limitations of Existing Tools

A second, orthogonal problem concerns the structural mismatch between the data model targeted by RML and the rendering model assumed by existing form builders.

The single-root assumption. Each of the SHACL-based form tools surveyed in this thesis shares a common architectural assumption: a form renders exactly one root node. This assumption is understandable. HTML forms are inherently hierarchical: a form is rendered as a sequence of nested elements, and no native HTML mechanism allows two distinct sections of a form to share a common sub-section without duplicating it. The browser rendering model is a tree. The natural mapping from SHACL to HTML therefore starts from a single `sh:NodeShape`—identified by a shape URI passed as a configuration parameter—and expands it recursively into a tree of form sections.

The problem is that RDF is not a tree. An RML mapping graph may contain multiple `TriplesMap` resources at the same level, none of which is necessarily subordinate to the other [2]. In the tested tools, the only available workaround is to introduce an artificial root node that aggregates all top-level resources as properties—an approach that departs from the official RML-core vocabulary and requires additional shape engineering. This is precisely the solution adopted in this thesis (see Section 3.3), and its necessity is a direct consequence of the single-root limitation.

Node reuse across the graph. The mismatch runs deeper than the number of root nodes. In an RML mapping, two `TriplesMap` resources may share the same `LogicalSource`—for instance, when two mappings read from the same CSV file. In a tree-based form, such a shared node either has to be duplicated at each point of use—introducing the risk of inconsistency if the user modifies one copy but not the other—or the form must provide a mechanism to create a node once and reference it from multiple parent sections.

The ULB-Darmstadt *shacl-form* does provide a partial mechanism for this: the `setClassInstanceProvider` callback allows the form to propose a list of existing graph instances for a given class, enabling the user to select and reuse a pre-existing node [15]. However, empirical testing reveals that this mechanism only operates over nodes that already exist in the data graph before the form session begins. It does not support referencing a node that was *created during the same form session*—which is precisely the scenario that arises when a user authors a complete RML mapping from scratch, creating a `LogicalSource` in one section and immediately reusing it in another. This gap is unsurprising: the typical use case for a generic SHACL form tool is describing one independent resource—one person, one address, one event—where node sharing within a single session does not arise. The scenario of authoring an interconnected RML mapping graph from scratch is outside that scope.

This gap motivates one of the design decisions presented in Section 3.3.

Why existing tools do not address this. The single-root constraint and the absence of node-reuse mechanisms are not oversights in the surveyed tools: they reflect the scope for which those tools were designed. The ULB-Darmstadt *shacl-form* library, for instance, requires a root shape URI to be passed as a configuration parameter [15]; empirical testing confirms that the form renders exactly the subgraph rooted at that node. No mechanism is provided for authoring multiple independent root nodes within a single editing session. This is a reasonable design choice for the common use case — describing one person, one address, one event — but becomes a limitation when the target data model is genuinely graph-shaped rather than tree-shaped. This thesis addresses precisely that gap.

3.2 Design Requirements

The two structural problems identified above — the expressivity boundary of SHACL with respect to form generation, and the graph-versus-tree mismatch — translate directly into five design requirements.

R1 — Statically interpretable shape profile. The shapes used to drive the form must contain only SHACL constructs that are *statically interpretable* (see Section 3.1.1): constructs whose contribution to widget selection, cardinality, and structural branching can be determined from

the shape definition alone. Constructs that are meaningful for validation but not statically interpretable (e.g. `sh:sparql`, `sh:alternativePath`) must be rewritten or excluded from the form profile.

R2 — Multi-root support. The form must be able to manage multiple top-level RDF resources within a single editing session. In the RML context, this means that the user must be able to create several `TriplesMap` resources without submitting the form between each one.

R3 — Node reuse. The form must allow a resource created in one part of the graph to be referenced from another part, without duplicating the resource's data. This is the mechanism that transforms tree-shaped authoring into graph-shaped authoring.

R4 — Explicit alternatives. When a property may take values of structurally incompatible types (expressed via `sh:or` or `sh:xone`), the form must present the choice explicitly — rendering exactly one branch at a time based on user selection —rather than exposing all branches simultaneously. Note that this requirement applies to *typed* alternatives with unambiguous write semantics; path-level ambiguity such as `sh:alternativePath` is addressed separately by R1.

R5 — Shape annotations. The shape profile must be augmentable with interaction-oriented annotations that do not interfere with RML semantics and do not affect conformance validation. This allows the form to be guided without altering the domain model.

These requirements directly answer the question of *what must be created*. The solution is not a new form library from scratch. Rather, it is (i) a methodology for writing SHACL shapes in a way that is form-friendly, and (ii) a set of extensions to an existing tool that implement R2 and R3. The next sections develop each component of this solution.

% TODO : modifier le terme " form-friendly " par un meilleur terme.

Table 3.1 situates the surveyed tools with respect to these requirements, and motivates the need for the contributions presented in the following sections.

	R1	**R2**	**R3**	**R4**	**R5**
ULB-Darmstadt *shacl-form*	○	×	○	✓	×
CSIRO *shacl-form*	○	×	×	○	×
Shaperone	○	×	×	○	×

Table 3.1: Coverage of design requirements by surveyed tools. ✓ = supported, ○ = partial, × = not supported.

3.3 The Form-Oriented Shape Profile (R1 + R5)

In the RML use case, official SHACL shapes are already available. The first question is therefore whether these shapes can be reused directly for form generation, or whether they must be rewritten. Requirements R1 and R5 provide the answer. The shapes driving the form must, first, contain only constructs that are statically interpretable by the form engine and, second, be augmentable with interaction-oriented annotations that improve the usability of the form without altering the semantics of the generated mapping.

This observation leads to a simple methodological choice. If shapes are written from scratch, they should be designed from the outset to satisfy these two requirements. If shapes already exist, as in the case of RML-core, they must be examined and transformed so that they satisfy them as well. In both cases, the result is what we call a **form-oriented shape profile**.

3.3.1 Why the official shapes are not sufficient

The official RML-core SHACL shapes [3] are designed to validate RML mappings. They are therefore exhaustive, semantically precise, and closely aligned with the specification. However, these same qualities make them difficult to use directly as a form specification. A validation shape may express what is allowed in RDF without necessarily indicating how that constraint should be rendered in an interface.

Several constructs found in or around the official shapes illustrate this gap. A first example is `sh:alternativePath`, which is used to express that several predicates are semantically acceptable, such as a shortcut property and its long-form equivalent. This is useful for validation, but ambiguous for form generation, because a form must decide which predicate it will actually write. A second example is the use of abstract and inherited shapes through `sh:and`, which captures reusable validation structure but also creates additional levels of nesting that do not necessarily correspond to meaningful interface sections for the user. A third example concerns advanced targeting mechanisms such as `sh:targetSubjectsOf` and `sh:targetObjectsOf`. These are useful to identify focus nodes in an existing RDF graph, but they do not provide a practical entry point for initializing a form whose goal is to create new data. Finally, some constraints are simply validation-oriented and do not help the form generator decide what widget or interaction pattern should be displayed.

The conclusion is not that the official shapes are poorly designed. On the contrary, they are well suited to validation. The problem is that validation completeness and interface interpretability are not the same objective. A dedicated profile is therefore needed to bridge this gap.

3.3.2 What a profile is

In this thesis, the term *profile* does not refer to a new language. It refers to a Turtle document containing SHACL shapes that have been selected, rewritten, and enriched for the specific purpose of form generation. The profile remains a SHACL shapes graph, but it is no longer intended to be a direct copy of the official validation shapes. Its purpose is different: not to express every possible validation rule, but to provide a shape set that can be interpreted reliably by the form engine.

A form-oriented shape profile can therefore be understood as an operational subset and adaptation of a richer validation model. It preserves the intended modelling semantics of the domain, but recasts them into a form that better supports widget selection, structural decomposition, and interaction design. For the present work, this means deriving a form-oriented RML profile from the official RML-core shapes.

3.3.3 How shortcuts and alternative paths are rewritten

One of the central rewriting tasks concerns properties that can be expressed in more than one equivalent way in RDF. RML contains several such cases. For instance, the shortcut `rml:subject` is semantically equivalent to a more explicit long form using `rml:subjectMap` together with `rml:constant`. In the validation shapes, this kind of equivalence may be represented using `sh:alternativePath`.

For form generation, however, such equivalence cannot remain implicit. A form must know which RDF structure it will produce. Two rewriting strategies are therefore possible. When the alternatives correspond to a meaningful user choice, the ambiguity can be made explicit in the interface, for example by rewriting the shape into a branching structure using `sh:xone` or `sh:or`. When the alternatives are only two notational variants of the same intent, the profile should commit to one canonical authoring form.

In the RML use case, the second strategy is often preferable. The form profile can simply keep the long form and omit the shortcut from the interface. This does not remove any semantic expressivity from the mapping; it only fixes one writing convention for the sake of clarity and predictable rendering. The same principle applies more generally: every place where the official shapes leave several equivalent RDF encodings open, the form-oriented profile must reduce this multiplicity to one renderable structure or one explicit user choice.

3.3.4 How UI triples are kept separate from domain triples

A second design consequence of R5 is that the profile may contain information that exists only to guide the interface. This raises an important question: if the form is driven by shapes augmented with UI-oriented information, how can one ensure that these additional elements do not pollute the final exported mapping?

The solution adopted here is to distinguish conceptually between a **domain layer** and a **UI layer**. The domain layer contains the shapes that describe the RML concepts to be authored, such as `TriplesMap`, `LogicalSource`, or `SubjectMap`. The UI layer contains additional elements used only to organise or improve the form, such as a wrapper root shape, ordering hints, grouping information, or other presentation-oriented metadata.

This separation does not require a different formalism. Both layers are still written in Turtle and still use SHACL-compatible constructs. The distinction is methodological: some triples belong to the domain model, while others belong to the form configuration. At export time, only the domain RDF graph is retained as output. UI-related triples are filtered out, so that the resulting serialisation contains only the RML mapping itself.

This approach is compatible with RML-core. The specification allows an RML mapping graph to contain additional triples outside the RML vocabulary, which are ignored by RML processors [3]. This makes it possible to use auxiliary resources for form generation without invalidating the mapping. Nevertheless, from the user's point of view, these auxiliary triples should not appear as if they were part of the authored mapping. The separation therefore matters not only semantically, but also for the clarity of the user experience.

3.3.5 Annotations for interaction

Once the profile has been restricted to statically interpretable constructs, it can be enriched with annotations that improve the usability of the form. These annotations do not change the RDF semantics of the mapping; they only improve how the form is presented and filled in. Relevant annotations include human-readable labels, field descriptions, explicit ordering, grouping of related fields, and default values for frequent cases. In practice, this means using properties such as `sh:name`, `sh:description`, `sh:order`, `sh:group`, and, where supported, `sh:defaultValue`.

This list is not intended to be exhaustive. The point is not to enumerate all possible UI-oriented predicates, but to show that the profile may legitimately contain declarative metadata that make the form clearer and more efficient to use. In the RML setting, these annotations are particularly useful because the same technical concept may otherwise appear opaque to non-expert users. Adding a short label, a description, or a sensible default can substantially reduce the cognitive cost of authoring a mapping through a form.

3.4 The Tree-Graph Gap and Node Reuse (R2 + R3)

Requirements R2 and R3 address a structural challenge that is independent of the shape vocabulary: the mismatch between the hierarchical rendering model of HTML forms and the graph-based data model of RDF. This section explains where this mismatch comes from, why it matters in the RML context, and what mechanism is proposed to address it.

3.4.1 How form engines render shapes

A SHACL-based form engine works by starting from a root node shape and recursively expanding each `sh:node` property into a nested sub-form. The result is a tree: each sub-form section has exactly one parent section, and the form as a whole is a hierarchical structure rooted at a single entry point.

This rendering model is, in one hand, a design assumption of existing tools: they are built to edit one resource at a time, not to author interconnected graphs. On the other hand, it is also a direct consequence of how HTML works: the DOM is a tree. A form is rendered as a sequence of nested elements, and no native HTML mechanism allows two distinct sections of a form to share a common

sub-section without duplicating it. This is not a limitation of any particular form library; it is a structural constraint of the web rendering model.

The consequence for form generation is that a form engine, by default, produces a tree-shaped representation of the data, even when the underlying data model is a graph.

3.4.2 Why this matters for RML

An RML mapping is represented as an RDF graph, not a tree. In practice, this means that several resources in a mapping may legitimately point to the same third resource. The most common case is `LogicalSource` reuse: two or more `TriplesMap` resources reading from the same source file will reference the same `LogicalSource` node. More generally, any shared configuration — a `SubjectMap` reused across mappings, a reference formulation shared by several iterables — creates this kind of convergence in the graph.

In a tree-based form, this situation cannot be represented directly. The options available are either to duplicate the shared resource at each point of use, which introduces inconsistency risks when the user later edits one copy but not the others, or to not expose reuse at all, which forces the user to manage shared structure manually outside the form. Neither option is satisfactory for a tool whose goal is to guide the user through the authoring of a complete and consistent mapping.

3.4.3 A session-level node registry

The proposed solution introduces a **session-level node registry**: a data structure maintained by the form throughout the authoring session. Each time the user creates a named resource through the form, that resource is registered in the registry with its IRI, its type, and optionally a display label.

When the form encounters a property whose value should be a resource — indicated by `sh:class` or `sh:node` in the shape — it presents the user with two options instead of one:

1. **Create new**: instantiate a fresh resource and open a sub-form for it, populating the registry as a side effect;

2. **Link existing**: select a previously created resource of a compatible type from the registry, without opening a sub-form.

Option (2) is what makes graph-shaped authoring possible within a tree-shaped rendering model. The sub-form is not duplicated; instead, the form records a reference to an already-existing node. The result, at the RDF level, is a genuine shared resource — one node pointed to by multiple parents — rather than two independent copies.

The registry is entirely in-memory and scoped to the current form session. It does not require a backend, a database, or any pre-loaded data. This is what distinguishes it from the `setClassInstanceProvider` mechanism available in the ULB-Darmstadt tool [15], which enables selection from instances that already exist in the data graph before the session begins. In our case, the graph is being authored from scratch: no instances exist before the session starts, so the registry must be built dynamically as the user fills in the form.

3.4.4 Why this is the right level of abstraction

It is worth noting what the registry does *not* do. It does not implement a full graph editor. It does not allow the user to navigate the graph arbitrarily, edit nodes out of context, or define cyclic references. Its scope is deliberately narrow: it supports the specific pattern of *creating a node once and referencing it from multiple places*, which is precisely the pattern that arises in RML mappings with shared logical sources or shared subject maps.

This narrowness is a feature, not a limitation. A full graph editor would reintroduce the complexity that the form-based approach is designed to avoid. The registry provides just enough graph expressivity to handle the cases that matter in practice, without compromising the guided, linear authoring experience that makes forms accessible to non-expert users.

3.5 Handling Specific SHACL Constructs (R4)

Requirement R4 concerns SHACL constructs that offer a choice between alternative RDF structures — branches whose expected predicates and sub-shapes differ, such that only one can be meaningfully instantiated at a time. This section describes how each of these constructs is handled in the form engine, and closes with a systematic classification of all SHACL Core constructs with respect to their treatment in this work.

3.5.1 `sh:xone` and `sh:or`: structural alternatives

`sh:or` and `sh:xone` enumerate a list of shapes, and the value of the property must conform to at least one of them (`sh:or`) or to exactly one of them (`sh:xone`). In the RML profile, the typical use case is a `TermMap` that can be either a `constant`, a `reference`, or a `template` — three structurally distinct configurations.

The challenge for form generation is that these constructs introduce a branching point in the shape graph. A naive rendering strategy would display all branches simultaneously, showing the fields of every alternative at once. This is problematic for two reasons. First, it is visually confusing: fields that are mutually exclusive appear as if they were all expected. Second, it produces ambiguous RDF output: if the user fills in fields from multiple branches, the resulting triples may conform to more than one alternative, which violates the intent of `sh:xone`.

The solution adopted here is to render both constructs as an **explicit choice widget**: a dropdown placed above the branching section. The user first selects one alternative, and only the fields of that alternative are then displayed. Switching to another alternative is always possible via a reset mechanism,with a warning if data would be lost. This design makes the mutually exclusive nature of the alternatives immediately visible.

Unlike the ULB-Darmstadt implementation, in which a choice made under `sh:or` cannot be undone without deleting the entire section, the present design provides a consistent reset mechanism for both `sh:xone` and `sh:or` [15]. This reflects the principle that authoring errors should always be recoverable without requiring the user to delete and recreate an entire mapping section.

At the interface level, `sh:or` and `sh:xone` are rendered identically. The semantic difference between them is enforced at validation time on the resulting RDF, not during form entry. In the RML form profile developed for this thesis, the branching cases encountered are mostly instances of mutually exclusive alternatives. The appropriate construct for these cases is therefore `sh:xone`. `sh:or` remains supported by the form engine.

3.5.2 `sh:in`: value enumeration

`sh:in` constrains a property to a fixed set of literal values or IRIs. In the RML context, this appears for properties such as `rml:referenceFormulation`, whose value must be one of a small set of well-known IRIs (`ql:CSV`, `ql:JSONPath`, `ql:XPath`).

Unlike `sh:xone`, `sh:in` does not open a sub-form after selection: the dropdown directly produces the chosen value as the object of the triple. The widget is therefore simpler — a plain select element — but the rendering principle is the same: the fixed list of options is read directly from the shape, and no free-text input is offered.

3.5.3 `sh:languageIn`: language tag enumeration

`sh:languageIn` constrains a literal value to a fixed set of language tags, such as `["en", "fr", "de"]`. Empirical testing on the ULB-Darmstadt form engine shows that this construct is rendered as a text field augmented with a language flag indicator on the right, which opens a dropdown of the permitted language tags when clicked.

This interaction pattern is well-suited to the construct: it keeps the literal value and its language tag visually associated, and makes the permitted languages discoverable without occupying a separate form row. The same pattern is retained in the present design.

3.5.4 Relationship to sh:alternativePath

It is important not to confuse sh:xone and sh:or with sh:alternativePath. The latter is a path expression, not a constraint: it states that several predicates are acceptable for the same property slot. It does not describe alternative structures, but alternative ways to write the same predicate.

Consider the following example from the RML-core shapes:

```
ex:SubjectMapShape sh:property [
    sh:path [ sh:alternativePath ( rml:subjectMap rml:subject ) ] ;
    sh:maxCount 1 ;
] .
```

This shape states that the value may be reached via either rml:subjectMap or the shortcut rml:subject. Both paths lead to the same semantic result: no structural choice is involved. By contrast, sh:xone would be used when the alternatives differ in structure, as in the following:

```
sh:xone (
    [ sh:property [ sh:path rml:constant ; sh:maxCount 1 ] ]
    [ sh:property [ sh:path rml:reference ; sh:maxCount 1 ] ]
    [ sh:property [ sh:path rml:template ; sh:maxCount 1 ] ]
) .
```

Here, the user must choose one structural variant, not one predicate name.

As discussed in Section 3.3, sh:alternativePath is handled by rewriting it to a canonical predicate in the form profile. sh:xone and sh:or, by contrast, are directly supported by the form rendering strategy described in this section.

3.5.5 Systematic classification of SHACL Core constructs

Not all SHACL Core constructs require dedicated treatment in a form engine. Table 3.2 classifies constructs with respect to their role in this work. The list is complete with respect to SHACL Core, but the treatment column onlycovers the constructs that appear in the RML form profile.

Table 3.2: SHACL Core constructs and their treatment in this work.

Construct(s)	Category	Treatment
sh:xone, sh:or	Structural alternatives	Explicit choice widget; this section
sh:in	Value enumeration	Dropdown (literal/IRI); this section
sh:languageIn	Language enumeration	Text + flag widget; this section
sh:alternativePath	Predicate ambiguity	Rewritten to canonical predicate; §3.3
sh:and, sh:node	Nesting / inheritance	Flattened in form profile; §3.3
sh:minCount, sh:maxCount	Cardinality	Standard rendering (add/remove buttons)
sh:datatype, sh:class	Type constraints	Widget selection (text, IRI, date...)
sh:not	Negative constraint	Not renderable; validation only
sh:closed	Closed world	Validation only; no form impact
sh:sparql	SPARQL rules	Out of scope; absent from RML profile
sh:pattern, sh:minLength, sh:maxLength, ...	String constraints	Inline validation; no dedicated widget

Constructs in the lower part of the table are either validation-only (they constrain the RDF graph but do not inform widget selection), out of scope for the RML profile, or handled as standard behaviour by anycompliant form engine. The constructs treated explicitly in this section are those that require a non-trivial rendering decision: they are not reducible to a plain text input or a simple add/remove button, and their incorrect rendering would produce ambiguous or invalid RDF output.

3.6 Interface Design Principles and Mockups

Chapter 4

Evaluation

Chapter 5

Conclusion and Future Work